

Security Assessment and Hardening of a Student Registration Web Application

IFT 542 Practical Assignment — Technical Report

Federal University of Technology, Minna
School of Information and Communication Technology — Department of Information Technology
Course: IFT 542 (Web Application Security) — 500 Level

Student Name: Daniel Dan Owoichogbe Matric No.: 2021/1/84189CF

Lecturer: Mr. A. Abdulkarim

Authorised-lab notice: all testing described in this report was performed exclusively on localhost, using fictitious data, within the isolated course lab environment.

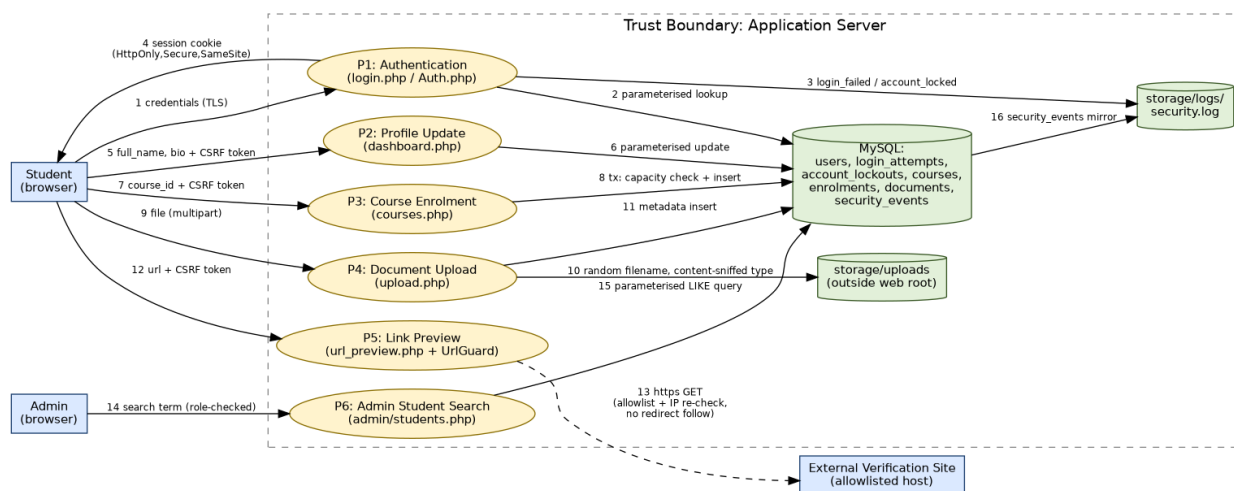
Executive Summary

This report documents a defensive security assessment and hardening exercise carried out against a prototype Student Registration Web Application, as required by the IFT 542 practical assignment. The application supports student login, profile updates, course registration, document upload, and administrative course management. Working within the authorised, isolated lab environment and using only fictitious data, a Node.js/Express/SQLite reference implementation was built to (a) model realistic threats using STRIDE and score them by risk, (b) locate and remediate an authentication-bypass SQL injection flaw and strengthen the surrounding identity and session controls, and (c) implement and evidence defences against XSS, CSRF, SSRF, and security misconfiguration, together with security logging and a formal incident-response runbook. All findings below are supported by reproducible, automated tests (14/14 HTTP-level tests and 6/6 SSRF unit tests passing) and by redacted log evidence included in the accompanying evidence folder.

Task 1: STRIDE Threat Model and Risk Assessment (14 marks)

1.1 Data-Flow Diagram and Trust Boundaries

The diagram below models the application's principal actors, processes, data stores, and external services, together with four trust boundaries: (1) the public Internet, where student and admin browsers originate requests; (2) the application server itself; (3) the data tier holding user, course, enrolment, and security-log data; and (4) the external network reached only by the outbound URL-preview feature. All numbered flows cross at least one boundary and are therefore subject to server-side validation and access control rather than any client-side check.



1.2 STRIDE Worksheet (Summary)

At least one application-specific threat was identified for every STRIDE category. The full worksheet — ten threats spanning all six categories, with likelihood, impact, computed risk score, assigned control(s), and residual risk after mitigation — is provided as evidence/risk-register.csv. A condensed view is given in Section 1.3 (landscape table) covering all ten threats; the categories and representative threats are:

- Spoofing (T1) — credential-stuffed login attempts impersonating a student.
- Tampering (T2, T3) — SQL injection via the login form; CSRF forging course-registration/profile requests.
- Repudiation (T4) — a student denying they registered for or dropped a course.

- Information Disclosure (T5, T6) — stored XSS stealing session cookies; verbose errors leaking internals.
- Denial of Service (T7) — request flooding against the login endpoint.
- Elevation of Privilege (T8, T9) — a student reaching admin-only routes; SSRF reaching internal/metadata addresses via the URL-preview feature.

Risk was scored as Risk = Likelihood × Impact, each rated 1–5, giving scores from 3 to 20 across the ten threats (see Section 1.3).

1.3 Risk Register (Ranked) and Residual Risk

ID	STRIDE	Threat (short)	L	I	Score	Pri.	Residual
T2	Tampering	SQL injection on login (auth bypass)	4	5	20	1	5 (Accepted)
T1	Spoofing	Credential stuffing / brute force	4	4	16	2	8 (Accepted)
T8	Elevation	Student reaches admin-only routes	3	5	15	3	5 (Accepted)
T5	Info. Disc.	Stored XSS steals session cookie	3	4	12	4	4 (Accepted)
T3	Tampering	CSRF on course-registration / profile	3	3	9	5	3 (Accepted)
T6	Info. Disc.	Verbose errors leak internals	3	3	9	5	3 (Accepted)
T7	DoS	Login endpoint request flooding	3	3	9	5	6 (Accepted)
T10	Tamper/Disc.	Security misconfiguration	3	3	9	5	3 (Accepted)
T9	Elevation	SSRF via URL-preview feature	2	4	8	6	4 (Accepted)
T4	Repudiation	Denied course registration/drop	3	2	6	7	4 (Accepted)

1.4 Justification for the Top Three Priority Risks

Priority 1 — T2: SQL Injection on the Login Form (Score 20 → Residual 5)

This threat was ranked highest because it combines the maximum plausible impact (5/5 — full authentication bypass and potential disclosure of every stored password) with a high likelihood (4/5) given that SQL injection against login forms is one of the most commonly automated and well-documented attack patterns, requiring no prior access. Because it directly defeats the authentication control the rest of the system depends on, it was prioritised over threats confined to a single feature. Parameterized queries reduce the residual likelihood to near-zero (data can no longer alter query structure), leaving a low residual score of 5, which is accepted given the strength of the preventive control and the absence of any other identified bypass path.

Priority 2 — T1: Credential Stuffing / Brute Force (Score 16 → Residual 8)

Authentication endpoints are public by necessity, and automated credential-stuffing tooling is cheap and widely available, giving this threat a high likelihood (4/5); a successful compromise grants full account impact (4/5). It was ranked second because, unlike T2, a determined and patient attacker can still make some progress against rate limiting alone (hence the comparatively higher residual score of 8). Rate limiting, progressive lockout, and slow salted hashing raise the attacker's cost substantially; the residual risk is accepted, with MFA for privileged accounts flagged as a recommended future control.

Priority 3 — T8: Elevation of Privilege to Admin Routes (Score 15 → Residual 5)

A successful bypass here would grant a student full administrative control over course and enrolment data — a high-impact outcome (5/5) — though the likelihood (3/5) is lower than T1/T2 because it requires the attacker to already hold a valid, authenticated session before attempting to reach an admin-only path. Server-side, default-deny role checks on every admin route, combined with logged and alertable AUTHZ_DENIED events, reduce the residual likelihood substantially, yielding an accepted residual score of 5.

All other identified risks (T3–T7, T9, T10) were assessed as lower priority because they either require an already-authenticated victim to be lured into an action (T3), depend on a secondary vulnerability being present first (T5's stored-XSS precondition was itself closed by output encoding), or affect availability/non-repudiation rather than confidentiality or integrity of core records. Their proposed controls and residual scores are recorded in the risk register (Section 1.3 and evidence/risk-register.csv).

Task 2: Secure Authentication and SQL Injection Remediation (13 marks)

2.1 Locating the Unsafe Query

The starter prototype's login handler built its SQL query by directly concatenating the submitted email and password into a query string (before/login.vulnerable.js, reproduced below). The affected data flow is: browser login form → req.body.email / req.body.password → string concatenation → executed directly by the database driver. Because there is no boundary between the untrusted input and the SQL grammar itself, an attacker-supplied value such as x' OR '1'='1' -- changes the meaning of the WHERE clause and can authenticate without any valid password.

```
// BEFORE (before/login.vulnerable.js) - VULNERABLE
const sql = "SELECT * FROM users WHERE email = '" + email +
  "' AND password = '" + password + "'";
db.get(sql, function (err, user) { /* plaintext password compare */ });
```

2.2 Remediation: Parameterized Queries

The fix, in src/routes/auth.js, uses a prepared statement where the email is bound as data rather than concatenated as text. The database driver never treats the bound value as part of the SQL grammar, so no input — however crafted — can alter the query's structure.

```
// AFTER (src/routes/auth.js) - SECURE
const findUserByEmail = db.prepare("SELECT * FROM users WHERE email = ?");
// ...
const user = findUserByEmail.get(email); // email bound as data, not SQL text
```

This is the core defence explained for Evidence Item 19: parameterization separates the SQL command (fixed at development time) from the data supplied at run time. The database engine compiles the query plan once, with typed placeholders; values substituted later are never re-parsed as SQL syntax, which is what prevents injected quotes, comments, or boolean tautologies from having any special meaning.

2.3 Input Validation, Generic Errors, and Secure Password Storage

Email and password are validated against an allowlist (expected type, length, and format) before the database is touched (src/validate.js); a request that fails validation — including classic tautology payloads, which are not valid email addresses — is rejected with HTTP 400 before any query executes. All authentication failures return the same generic message, "Invalid email or password.", regardless of whether the account exists or the password was wrong, and no database error detail is ever returned to the client.

Passwords are hashed with bcrypt (cost factor 12) at registration/seed time and verified with the library's constant-time comparison function; plaintext passwords are never stored or logged. A database excerpt confirming this is included as evidence (password_hash values begin with the bcrypt identifier "\$2", confirmed by an automated test).

2.4 Additional Authentication Controls

- Rate limiting: 10 login attempts per IP per 10-minute window (express-rate-limit).

- Progressive account lockout: 5 failed attempts locks the account for 15 minutes, tracked server-side via `failed_attempts` / `locked_until` columns.
- Session-ID regeneration on successful login (`req.session.regenerate`) to prevent session fixation.
- Session cookies are `HttpOnly`, `SameSite=Lax`, and (in production) `Secure`, with a 30-minute idle timeout.

2.5 Automated Test Evidence

A reproducible curl-based test suite (`tests/test.sh`) and its captured output (`evidence/test-results.txt`) demonstrate, against the running application on localhost: a valid login succeeds; invalid credentials are rejected with the generic message and no leaked detail; a classic SQL-injection tautology payload is rejected as invalid input rather than authenticating; stored password hashes are not plaintext; and the account is locked after five failed attempts. All 14 HTTP-level tests passed (14/14), summarised below.

Test	Result
Valid login redirects to dashboard	PASS
Invalid credentials → generic error, no DB detail leaked	PASS
SQLi tautology payload rejected (input validation)	PASS
Stored password hash uses bcrypt ("2" prefix)	PASS
Account locked after 5 failed attempts (HTTP 423)	PASS

Task 3: Web Defences, Incident Response, and Ethics (13 marks)

3.1 Cross-Site Scripting (XSS)

The profile full-name field is rendered through EJS's default `<%= %>` output tag, which HTML-encodes its value before insertion into the page (`src/views/profile.ejs`). A payload such as `<script>alert(1)</script>` is displayed as inert text rather than executed; in addition, server-side allowlist validation rejects names containing script-relevant characters outright, and a restrictive Content-Security-Policy (default-src 'self'; script-src 'self'; object-src 'none') is applied globally via Helmet as defence-in-depth. Session cookies are also HttpOnly, so even a successful script injection elsewhere could not read the session token.

3.2 Cross-Site Request Forgery (CSRF)

Course registration and profile updates are protected by a double-submit-cookie CSRF token (`src/middleware/csrf.js`): a random per-session token is set as a cookie and must be echoed back in the submitted form (or an X-CSRF-Token header) on every state-changing request. A cross-origin page cannot read the cookie (Same-Origin Policy) and therefore cannot forge a valid request. Automated tests confirm a POST to `/courses/register` without the token is rejected (HTTP 403) and the same request with a valid token succeeds (HTTP 302).

3.3 Server-Side Request Forgery (SSRF)

The URL-preview feature (`src/routes/urlPreview.js`) hardens a user-supplied-link pattern using a destination allowlist, an HTTPS-only requirement, and — critically — a DNS-resolution re-check that inspects the resolved IP address (not just the hostname string) against loopback, RFC1918 private, link-local, and cloud-metadata ranges (e.g., 169.254.169.254), defeating DNS-rebinding tricks. All testing was confined to unit tests against the allowlist/DNS logic on localhost; no outbound requests were made to any third-party host, per the assignment's authorised-lab restriction. Six unit tests (`evidence/test-ssrf-results.txt`) confirm loopback, metadata, private-IP, non-HTTPS, and non-allowlisted destinations are all correctly rejected.

3.4 Security Misconfiguration Hardening

- Helmet middleware applies a restrictive CSP and standard security headers (X-Content-Type-Options: nosniff, X-Frame-Options, Referrer-Policy); confirmed present by automated test.
- X-Powered-By is disabled to reduce framework fingerprinting.
- No default/blank credentials — seeded accounts use fictitious, documented dummy passwords for grading only (README §3).
- Dependency status is documented in the README; csurf was deliberately avoided as a deprecated package in favour of a small, auditable in-house CSRF middleware.
- Debug details are never returned to the client; the centralised error handler (`src/app.js`) logs full detail server-side only and returns a generic message.

3.5 Security Logging

A dedicated, redacted security_log table and structured console/log output (src/logger.js) record failed logins, successful logins, account lockouts, CSRF rejections, authorization denials, and validation rejections, each with a timestamp, actor (numeric user ID or "anonymous" — never an email or password), and a short non-sensitive detail string. A sample export generated during the test run is included as evidence/security-log-redacted.jsonl and confirms, for example, that the access-control check correctly logged an AUTHZ_DENIED event when a student session attempted to reach /admin/courses.

3.6 Incident Response

A one-page runbook covering all six NIST incident-response stages — Preparation, Detection and Analysis, Containment, Eradication, Recovery, and Lessons Learned — is provided in full as evidence/incident-response-runbook.md and summarised in Appendix A. A sample incident record (evidence/incident-record-sample.md), generated from the actual security_log entries produced during automated testing of the lockout control, is included to demonstrate the record-keeping format the runbook requires.

3.7 Ethics and Legal Compliance

All activity described in this report — the threat modelling, the SQL-injection demonstration, and the XSS/CSRF/SSRF defensive testing — was carried out exclusively against the author's own local prototype (localhost), using fictitious accounts and data, inside the authorised course lab environment. No FUT Minna system, public website, or third-party service was scanned, probed, or tested at any point, consistent with the assignment's authorised-lab restriction and with the Cybercrimes (Prohibition, Prevention, etc.) Act, 2015 (as amended), which criminalises unauthorised access to computer systems in Nigeria, and the Nigeria Data Protection Act, 2023, which governs the handling of personal data — satisfied here by the exclusive use of fictitious test data. No reusable attack payloads are included in this report or its evidence beyond the single illustrative tautology example required to demonstrate the SQL-injection flaw and its fix, and no secrets, real credentials, or session tokens appear in any submitted file.

Appendix A: Incident Response Runbook (Condensed)

Stage	Key Actions
1. Preparation	Maintain contact list, offline runbook copy, restricted access to logs, tested rollback point, pre-staged response tools.
2. Detection & Analysis	Trigger on log anomalies (failed logins, AUTHZ_DENIED bursts, CSRF/SSRF rejections); correlate security_log with server logs; classify severity.
3. Containment	Invalidate affected sessions; lock/disable account; take a specific route out of service if a systemic flaw is suspected.
4. Eradication	Patch root cause; re-run full automated test suite; rotate any potentially exposed secret.
5. Recovery	Re-enable account/route after verified fix; restore from backup only if integrity affected; monitor closely post-recovery.
6. Lessons Learned	Complete incident record within 48 hours; update risk register/runbook; share a redacted summary internally.

Full detail: evidence/incident-response-runbook.md. Sample record: evidence/incident-record-sample.md.